

Reviewer Pack — Minimal Rank of Geometric Quantization over Tseitin Circuit ...

Entry c74b429ad3e4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 16:36:33 UTC

Minimal Rank of Geometric Quantization over Tseitin Circuit Width

Entry ID: c74b429ad3e4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 16:36:33 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Quantization) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

[‘For any Tseitin formula G on n variables, the minimal rank of its geometric quantization $\varphi(G)$ over C is at least $2^{\Omega(w(G))}$, where $w(G)$ is the width of G .’, ‘Here, the geometric quantization $\varphi(G)$ is defined as the set of linear transformations that preserve the Hermitian form of G ’s adjacency matrix up to a global phase factor.’, ‘For a Tseitin formula on an expander graph, this implies that a resolution refutation with length $\leq 2^{\Omega(w(G))}$ cannot exist.’]

Rationale (proposer’s reasoning):

[‘Geometric quantization has been used in quantum information theory to encode the structure of quantum states and operations. Its application to complexity theory could provide new insights into the hardness of Tseitin formulas.’, ‘If this conjecture holds, it would suggest that the geometric properties of Tseitin formulas play a crucial role in their resolution complexity.’, ‘This might lead to new algorithms or heuristics for solving SAT problems efficiently.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dcc5c01d1b11ab96

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all Tseitin formulas G with width $w(G)$, the ratio of the computed minimal rank of $\varphi(G)$ to $2^{\Omega(w(G))}$ is at least 0.5, and falsified if any formula has a ratio less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric quantization" AND "Tseitin circuit width" AND minimal rank" - "quantum information theory" AND Tseitin formula AND geometric quantization" - "expander graph" AND resolution refutation AND $2^{\Omega(w(G))}$

Top relevant hits considered: - [<http://arxiv.org/abs/2412.03444v1>] A new fidelity of quantum channel evolution and its geometric interpretation - [<http://arxiv.org/abs/2403.09252v1>] Reverse em-problem based on Bregman divergence and its application to classical and quantum information theory - [<http://arxiv.org/abs/2406.19060v3>] Semi-definite optimization of the measured relative entropies of quantum states and channels - [<http://arxiv.org/abs/2005.11589v3>] MaxSAT Resolution and Subcube Sums - [<http://arxiv.org/abs/1912.00534v2>] Exponential Resolution Lower Bounds for Weak Pigeonhole Principle and Perfect Matching Formulas over Sparse Graphs - [<http://arxiv.org/abs/2507.23008v2>] Exponential Lower Bounds on the Size of ResLin Proofs of Nearly Quadratic Depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def adjacency_matrix(formula, n):
        A = [[0] * n for _ in range(n)]
        for clause in formula:
            literals = set(clause.split(' | '))
            for literal in literals:
                if literal[0] == '~':
                    var = int(literal[2:]) - 1
                else:
                    var = int(literal[1:]) - 1
                A[var][var] = 1
        return A

    def gaussian_elimination(A):
        n = len(A)
```

```

    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    rank = 0
    for i in range(n):
        if any(A[i]):
            rank += 1
    return rank

def tseitin_formula(n, width):
    formula = []
    literals = [f'x{i+1}' for i in range(n)]
    for _ in range(width):
        clause = random.sample(literals, 2)
        formula.append(f'{clause[0]} | {clause[1]}')
    return formula

n = random.randint(5, 30)
width = random.randint(2, n//2)
formula = tseitin_formula(n, width)

A = adjacency_matrix(formula, n)
rank = gaussian_elimination(A)

expected_rank = 2 ** (width * math.log2(width))

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank >= expected_rank,
    "counterexample": "" if rank >= expected_rank else f"rank={rank}, expected={expected_rank}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571de152.py", line 86, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571de152.py", line 68, in run_trial
    rank = gaussian_elimination(A)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571de152.py", line 45, in gaussian_elimination
    factor = A[j][i] / A[i][i]
              ^^^^^
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test without errors to verify if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11923
2	preregistration	ollama_remote	glm4:latest	0	0	5485
3	novelty	ollama_remote	glm4:latest	0	0	4967

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	10582
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17096
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10089
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7766
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9265
9	judge	ollama_remote	glm4:latest	0	0	8535

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 85709 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c74b429ad3e4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c74b429ad3e4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c74b429ad3e4.tar.gz` (if generated)